

Output del modelo

Se construyó un **modelo de clasificación binaria** para estimar aceptación comercial.

El modelo trabaja con una combinación específica de **cliente + oferta + contexto del ofrecimiento**. Eso significa que, para calcular el score, usa varias variables agrupadas en tres bloques:

1. Variables del cliente

Describen quién es el cliente y su comportamiento histórico. Por ejemplo:

- antigüedad;
- tipo de cliente;
- si tiene móvil, hogar o internet hogar;
- consumo de datos;
- consumo de voz y SMS;
- facturación;
- mora;
- meses moroso;
- reclamos;
- uso de la app;
- actividad comercial;
- edad/rango de edad;
- elegibilidad para Movistar Total.

2. Variables de la oferta

Describen qué producto se está evaluando. Por ejemplo:

- oferta_id;
- tipo de oferta;
- precio mensual;
- si es Movistar Total;
- ahorro asociado;
- GB incluidos;
- características del bundle.

3. Variables del contexto

Describen cómo se realizó o se evaluaría el ofrecimiento. En el modelo actual entra, entre otras cosas, el **canal** asociado a la evaluación.

Por eso el modelo responde:

“Dadas las características de este cliente, esta oferta concreta y el contexto disponible, ¿qué probabilidad hay de que la combinación termine en aceptación?”

Ejemplo:

Cliente CLI000013 + Oferta OF021 + contexto/canal disponible → score = 0.7131

Eso significa una **probabilidad estimada de aceptación de 71.31%** según los patrones históricos.

El modelo es binario porque fue entrenado con:

- 1 = acepta
- 0 = rechaza

Pero para el proyecto se conserva principalmente la **probabilidad**, en vez de convertirla directamente a sí/no.

Output del scoring

Una vez entrenado el modelo, se aplica a las distintas ofertas que se quieren evaluar para cada cliente.

Por ejemplo, para un mismo cliente:

Oferta Score de aceptación

OF021	71.31%
OF020	71.04%
OF022	70.84%

Luego esas probabilidades se ordenan de mayor a menor y se genera un **ranking predictivo**:

1. OF021 → 71.31%
2. OF020 → 71.04%
3. OF022 → 70.84%

Así, el scoring no crea otra predicción distinta: **usa las probabilidades del modelo y las ordena para cada cliente.**

Actualmente quedaron generados:

- **99,039 clientes con score**
- **22 ofertas evaluadas por cliente**
- **2,178,858 combinaciones cliente–oferta puntuadas**
- **Top 3 por cliente**
- **297,117 registros en el Top 3**

CSV 1: scoring completo

Contiene todas las combinaciones cliente–oferta evaluadas.

Ejemplo de columnas:

cliente_id | oferta_id | nombre_oferta | tipo_oferta | es_movistar_total | elegible_mt |
score_aceptacion | ranking_predictivo

Sirve para que la siguiente parte del sistema tenga **todos los scores disponibles**, no solo las tres mejores ofertas.

Por ejemplo, un backend puede consultar:

“Dame todas las ofertas puntuadas para CLI000013”.

y luego aplicar las reglas comerciales correspondientes.

CSV 2: Top 3 por cliente

Contiene únicamente las **tres ofertas con mayor score predictivo para cada cliente**.

Ejemplo:

cliente_id	oferta_id	score	ranking
CLI000013	OF021	0.7131	1
CLI000013	OF020	0.7104	2
CLI000013	OF022	0.7084	3

Este archivo es más práctico para:

- una demo;
 - una interfaz de asesor;
 - un dashboard;
 - mostrar rápidamente las mejores alternativas predictivas.
-

Qué puede hacer específicamente el modelo

El modelo **sí puede**:

- recibir datos de un cliente y una oferta;
- calcular una probabilidad de aceptación;
- repetir el cálculo para varias ofertas;
- generar scores que luego puedan ordenarse;
- reutilizarse con nuevos datos compatibles;
- integrarse posteriormente en un backend/API;
- alimentar una plataforma o sistema NBO.

El modelo **no puede por sí solo**:

- decidir si una oferta está legal/comercialmente permitida;
- saber automáticamente si debe priorizarse MT por estrategia;
- aplicar consentimiento, horarios o frecuencia de contacto;
- decidir el mensaje final;
- calcular rentabilidad real;
- manejar rebate;
- ejecutar la campaña;
- decidir la recomendación comercial definitiva.

Esas decisiones las toma otra capa del sistema usando el score como input.

Cómo se integraría después en una plataforma

El modelo quedó guardado, por lo que no sería necesario abrir el notebook cada vez.

La lógica sería:

Plataforma / backend

↓

envía variables del cliente + oferta

↓

modelo entrenado

↓

devuelve score_aceptacion

↓

backend compara scores y aplica reglas

↓

interfaz muestra la recomendación al asesor

Entonces, la pieza de Data Science entrega básicamente:

datos → probabilidad → ranking

y la aplicación completa convierte ese ranking en una decisión comercial usable.